

1	Design an student class for reading and displaying the student information, the getInfo() and displayInfo() methods will be used respectively. Where getInfo() will be a private method <pre> class Student: def __init__(self, name=None, roll_number=None, age=None): self.name = name self.roll_number = roll_number self.age = age def getInfo(self): self.name = input("Enter student's Name: ") self.roll_number = input("Enter student's roll number: ") self.age = input("Enter student's age: ") def displayInfo(self): print("Student Information:") print("Name:", self.name) print("Roll No:", self.roll_number) print("Age:", self.age) # Creating a student object s = Student() s.getInfo() # Input student information s.displayInfo() # Display student information </pre>
2	Design a class Complex for adding the two complex numbers and show the use of constructor <pre> class Complex: def __init__(self, real=0, imaginary=0): self.real = real self.imaginary = imaginary def add(self, other): real_sum = self.real + other.real imaginary_sum = self.imaginary + other.imaginary return Complex(real_sum, imaginary_sum) def display(self): if self.imaginary >= 0: sign = "+" else: sign = "-" print(f"{self.real} {sign} {abs(self.imaginary)}i") # Create two complex numbers complex1 = Complex(3, 4) complex2 = Complex(1, -2) # Display the two complex numbers print("Complex Number 1:") complex1.display() print("Complex Number 2:") complex2.display() # Add the two complex numbers result = complex1.add(complex2) </pre>

	<pre># Display the result print("Sum of Complex Numbers:") result.display()</pre>
3	Write Python class to represent a bank account. The class contains account holder name, account number, type of account and balance amount as the data members. The member functions are as follows: to initialize the data members, to deposit an amount, to withdraw an amount after checking balance, and to display name and balance. <pre>class BankAccount: def __init__(self, account_holder_name, account_number, account_type, balance=0): self.account_holder_name = account_holder_name self.account_number = account_number self.account_type = account_type self.balance = balance def deposit(self, amount): if amount > 0: self.balance += amount print(f"Deposited \${amount}. New balance: \${self.balance}") else: print("Invalid deposit amount. Please enter a positive value.") def withdraw(self, amount): if amount > 0: if self.balance >= amount: self.balance -= amount print(f"Withdrew \${amount}. New balance: \${self.balance}") else: print("Insufficient balance. Withdrawal not allowed.") else: print("Invalid withdrawal amount. Please enter a positive value.") def display(self): print("Account Holder Name:", self.account_holder_name) print("Account Number:", self.account_number) print("Account Type:", self.account_type) print("Balance:", self.balance) # Example usage: account1 = BankAccount("John Doe", "12345", "Savings", 1000) account1.display() account1.deposit(500) account1.withdraw(200) account1.display()</pre>
4	Write Python class to represent a Cricketer. The class contains name of cricketer, team name and run as the data members. The member functions are as follows: to initialize the data members, to set run and display run. <pre>class Cricketer: def __init__(self, name, team_name): self.name = name self.team_name = team_name</pre>

	<pre> self.runs = 0 # Initialize runs to 0 def set_runs(self, runs): self.runs = runs def display_runs(self): print(f'{self.name} from {self.team_name} has scored {self.runs} runs.') # Example usage: # if __name__ == "__main__": cricketer1 = Cricketer("Virat Kohli", "India") cricketer1.set_runs(12000) cricketer1.display_runs() cricketer2 = Cricketer("Steve Smith", "Australia") cricketer2.set_runs(8500) cricketer2.display_runs() </pre>
5	Implement push and pop algorithms of stack using list. <pre> class Stack: def __init__(self): self.items = [] def push(self, item): self.items.append(item) def pop(self): if not self.is_empty(): return self.items.pop() else: raise IndexError("Pop from an empty stack") def is_empty(self): return len(self.items) == 0 def peek(self): if not self.is_empty(): return self.items[-1] else: raise IndexError("Peek from an empty stack") def size(self): return len(self.items) # Example usage: if __name__ == "__main__": stack = Stack() stack.push(1) stack.push(2) stack.push(3) print("Stack:", stack.items) print("Stack Size:", stack.size()) print("Peek:", stack.peek()) popped_item = stack.pop() print("Popped Item:", popped_item) </pre>

```
print("Stack after popping:", stack.items)
print("Is Stack empty?", stack.is_empty())
```

6 Implement a program to convert infix notation to postfix notation using stack.

```
class Conversion:
    def __init__(self, capacity):
        self.top = -1
        self.capacity = capacity
        self.array = []
        self.output = []
        self.precedence = {'+': 1, '-': 1, '*': 2, '/': 2, '^': 3}

    def isEmpty(self):
        if self.top == -1:
            return True
        else:
            return False

    def peek(self):
        if not self.isEmpty():
            return self.array[-1]
        else:
            return "$"

    def pop(self):
        if not self.isEmpty():
            self.top -= 1
            return self.array.pop()
        else:
            return "$"

    def push(self, op):
        self.top += 1
        self.array.append(op)

    def isOperand(self, ch):
        return ch.isalpha()

    def notGreater(self, i):
        try:
            a = self.precedence[i]
            b = self.precedence[self.peek()]
            return True if a <= b else False
        except KeyError:
            return False

    def infixToPostfix(self, exp):
        for i in exp:
            if self.isOperand(i):
                self.output.append(i)
            elif i == '(':
                self.push(i)
            elif i == ')':
                while not self.isEmpty() and self.peek() != '(':
                    a = self.pop()
                    self.output.append(a)
                if not self.isEmpty() and self.peek() != '(':
```

```

        return -1
    else:
        self.pop()
    else:
        while not self.isEmpty() and self.notGreater(i):
            self.output.append(self.pop())
        self.push(i)

    while not self.isEmpty():
        self.output.append(self.pop())

    result = "".join(self.output)
    print(result)

# Driver program to test above function
exp = "a+b*(c^d-e)^(f+g*h)-i"
obj = Conversion(len(exp))
obj.infixToPostfix(exp)

```

7 Implement a program to implement queue using list that performs following operations: enqueue, dequeue, display.

```

# Class to represent a node
class StackNode:
    # Constructor to initialize a node
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    # Constructor to initialize the root of the linked list
    def __init__(self):
        self.root = None

    def isEmpty(self):
        return True if self.root is None else False

    def push(self, data):
        newNode = StackNode(data)
        newNode.next = self.root
        self.root = newNode
        print(f'{data} pushed to the stack')

    def pop(self):
        if self.isEmpty():
            return float("-inf")
        temp = self.root
        self.root = self.root.next
        popped = temp.data
        return popped

    def peek(self):
        if self.isEmpty():
            return float("-inf")
        return self.root.data

```

Driver program to test the above class

	<pre> stack = Stack() stack.push(10) stack.push(20) stack.push(30) print(f"{stack.pop()} popped from the stack") print(f"Top element is {stack.peek()}") </pre>
8	Implement program to perform following operation on singly linked list: - Insert a node at the beginning of a singly linked list. - Insert a node at the end of a singly linked list. - Insert a node after the given node of a singly linked list. - Insert a node before the given node of singly linked list. - Delete a node from the beginning of a singly linked list. - Delete a node from the end of a singly linked list. - Count the number of nodes of a singly linked list. - Display content of singly linked list. <pre> class Node: def __init__(self, data): self.data = data self.next = None class SinglyLinkedList: def __init__(self): self.head = None def insert_at_beginning(self, data): new_node = Node(data) new_node.next = self.head self.head = new_node def insert_at_end(self, data): new_node = Node(data) if not self.head: self.head = new_node return current = self.head while current.next: current = current.next current.next = new_node def insert_after_node(self, prev_node, data): if not prev_node: print("Previous node is not in the list.") return new_node = Node(data) new_node.next = prev_node.next prev_node.next = new_node def insert_before_node(self, next_node, data): if not self.head: print("List is empty.") return if self.head.data == next_node: self.insert_at_beginning(data) return current = self.head while current.next: if current.next.data == next_node: new_node = Node(data) </pre>

```
        new_node.next = current.next
        current.next = new_node
        return
    current = current.next
    print(f"Node with data {next_node} not found in the list.")
```

```
def delete_at_beginning(self):
    if not self.head:
        print("List is empty.")
        return
    self.head = self.head.next
```

```
def delete_at_end(self):
    if not self.head:
        print("List is empty.")
        return
    if not self.head.next:
        self.head = None
        return
    current = self.head
    while current.next.next:
        current = current.next
    current.next = None
```

```
def count_nodes(self):
    count = 0
    current = self.head
    while current:
        count += 1
        current = current.next
    return count
```

```
def display(self):
    current = self.head
    while current:
        print(current.data, end=" -> ")
        current = current.next
    print("None")
```

```
# Example usage
if __name__ == '__main__':
    linked_list = SinglyLinkedList()
    linked_list.insert_at_beginning(3)
    linked_list.display()
    linked_list.insert_at_beginning(2)
    linked_list.display()
    linked_list.insert_at_end(4)
    linked_list.display()
    linked_list.insert_after_node(linked_list.head, 5)
    linked_list.display()
    linked_list.insert_before_node(4, 6)
    linked_list.display()
    linked_list.delete_at_beginning()
    linked_list.display()
    linked_list.delete_at_end()
    linked_list.display()
    print("Number of nodes:", linked_list.count_nodes())
```

9	Implement program to create and display circular linked list. <pre> class Node: def __init__(self, data): self.data = data self.next = None class CircularLinkedList: def __init__(self): self.head = None def append(self, data): new_node = Node(data) if not self.head: self.head = new_node new_node.next = self.head else: current = self.head while current.next != self.head: current = current.next current.next = new_node new_node.next = self.head def display(self): if not self.head: return current = self.head while True: print(current.data, end=" -> ") current = current.next if current == self.head: break print() # Example usage if __name__ == '__main__': circular_list = CircularLinkedList() circular_list.append(1) circular_list.append(2) circular_list.append(3) circular_list.display() print("Circular Linked List:") circular_list.display() </pre>
10	Implement program to perform following operation on doubly linked list : traversal, searching, adding and removing node. <pre> class Node: def __init__(self, data): self.data = data self.next = None self.prev = None class DoublyLinkedList: def __init__(self): </pre>


```
self.head = None
```

```
def append(self, data):  
    new_node = Node(data)  
    if not self.head:  
        self.head = new_node  
    else:  
        current = self.head  
        while current.next:  
            current = current.next  
        current.next = new_node  
        new_node.prev = current
```

```
def search(self, target):  
    current = self.head  
    while current:  
        if current.data == target:  
            return current  
        current = current.next  
    return None
```

```
def add_after(self, node_data, data):  
    current = self.search(node_data)  
    if current:  
        new_node = Node(data)  
        new_node.next = current.next  
        new_node.prev = current  
        if current.next:  
            current.next.prev = new_node  
        current.next = new_node
```

```
def remove(self, data):  
    node_to_remove = self.search(data)  
    if not node_to_remove:  
        print(f"Node with data {data} not found.")  
        return  
    if node_to_remove.prev:  
        node_to_remove.prev.next = node_to_remove.next  
    else:  
        self.head = node_to_remove.next  
    if node_to_remove.next:  
        node_to_remove.next.prev = node_to_remove.prev
```

```
def display(self):  
    current = self.head  
    while current:  
        print(current.data, end=" <-> ")  
        current = current.next  
    print()
```

```
# Example usage
```

```
if __name__ == '__main__':  
    doubly_linked_list = DoublyLinkedList()  
    doubly_linked_list.append(1)  
    doubly_linked_list.display()  
    doubly_linked_list.append(2)  
    doubly_linked_list.display()  
    doubly_linked_list.append(3)  
    doubly_linked_list.display()
```

	<pre> print("Doubly Linked List:") doubly_linked_list.display() search_result = doubly_linked_list.search(2) if search_result: print("Found:", search_result.data) else: print("Not found") doubly_linked_list.add_after(2, 4) print("After adding 4 after 2:") doubly_linked_list.display() doubly_linked_list.remove(3) print("After removing 3:") doubly_linked_list.display() </pre>
11	Implement a python program to search a particular element from list using Linear and Binary Search.. <pre> # Linear Search Function def linear_search(arr, target): for i in range(len(arr)): if arr[i] == target: return i return -1 # Binary Search Function (Iterative) def binary_search(arr, target): low = 0 high = len(arr) - 1 while low <= high: mid = (low + high) // 2 # Check if target is present at mid if arr[mid] == target: return mid # If target is greater, ignore the left half elif arr[mid] < target: low = mid + 1 # If target is smaller, ignore the right half else: high = mid - 1 return -1 # Main function to test both searches def main(): # List to search the element in arr = [12, 34, 45, 56, 67, 78, 89, 90] target = int(input("Enter the element to search: ")) print("Performing Linear Search...") index = linear_search(arr, target) </pre>

	<pre> if index != -1: print(f"Element found at index {index} using Linear Search.") else: print("Element not found using Linear Search.") # Ensure the array is sorted for binary search arr.sort() print("Performing Binary Search...") index = binary_search(arr, target) if index != -1: print(f"Element found at index {index} using Binary Search.") else: print("Element not found using Binary Search.") # Run the main function if __name__ == "__main__": main() </pre>
12	Implement Bubble sort algorithm. <pre> # Bubble Sort Function def bubble_sort(arr): n = len(arr) # Traverse through all elements in the list for i in range(n): # Flag to track if any swapping happens swapped = False # Traverse the array from 0 to n-i-1 # Last i elements are already in place for j in range(0, n-i-1): # Swap if the element found is greater than the next element if arr[j] > arr[j+1]: arr[j], arr[j+1] = arr[j+1], arr[j] swapped = True # If no elements were swapped, the array is sorted if not swapped: break # Main function to test the bubble sort def main(): arr = [64, 34, 25, 12, 22, 11, 90] print("Original array:", arr) bubble_sort(arr) print("Sorted array:", arr) # Run the main function if __name__ == "__main__": main() </pre>
13	Implement Selection sort and Insertion sort algorithm.

```
# Selection Sort Function
def selection_sort(arr):
    n = len(arr)

    # Traverse through all array elements
    for i in range(n):
        # Find the minimum element in remaining unsorted array
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j

        # Swap the found minimum element with the first element
        arr[i], arr[min_idx] = arr[min_idx], arr[i]

# Main function to test the selection sort
def main():
    arr = [64, 25, 12, 22, 11]
    print("Original array:", arr)

    selection_sort(arr)

    print("Sorted array:", arr)

# Run the main function
if __name__ == "__main__":
    main()
```

```
# Insertion Sort Function
def insertion_sort(arr):
    # Traverse through 1 to len(arr)
    for i in range(1, len(arr)):
        key = arr[i]

        # Move elements of arr[0..i-1], that are greater than key,
        # to one position ahead of their current position
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key

# Main function to test the insertion sort
def main():
    arr = [12, 11, 13, 5, 6]
    print("Original array:", arr)

    insertion_sort(arr)

    print("Sorted array:", arr)

# Run the main function
if __name__ == "__main__":
    main()
```

14

Implement Merge sort algorithm.

```
# Merge Sort Function
def merge_sort(arr):
    if len(arr) > 1:
        # Finding the middle of the array
        mid = len(arr) // 2

        # Dividing the array elements into 2 halves
        left_half = arr[:mid]
        right_half = arr[mid:]

        # Recursively sorting both halves
        merge_sort(left_half)
        merge_sort(right_half)

        # Merging the sorted halves
        i = j = k = 0

        # Copy data to temp arrays left_half[] and right_half[]
        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1

        # Checking if any element was left in the left_half
        while i < len(left_half):
            arr[k] = left_half[i]
            i += 1
            k += 1

        # Checking if any element was left in the right_half
        while j < len(right_half):
            arr[k] = right_half[j]
            j += 1
            k += 1

# Main function to test the merge sort
def main():
    arr = [38, 27, 43, 3, 9, 82, 10]
    print("Original array:", arr)

    merge_sort(arr)

    print("Sorted array:", arr)

# Run the main function
if __name__ == "__main__":
    main()
```

15

Implement Quick sort algorithm.

```
# Partition Function to place the pivot element at the correct position
def partition(arr, low, high):
    pivot = arr[high] # We can choose the last element as pivot
```

```

i = low - 1 # Pointer for the smaller element

# Traverse the array and rearrange elements
for j in range(low, high):
    if arr[j] < pivot:
        i += 1
        arr[i], arr[j] = arr[j], arr[i] # Swap

# Swap the pivot element with the element at i+1
arr[i + 1], arr[high] = arr[high], arr[i + 1]

return i + 1 # Return the partitioning index

# Quick Sort Function
def quick_sort(arr, low, high):
    if low < high:
        # Partition the array and get the pivot index
        pi = partition(arr, low, high)

        # Recursively apply quick sort to the sub-arrays
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

# Main function to test the quick sort
def main():
    arr = [10, 7, 8, 9, 1, 5]
    print("Original array:", arr)

    # Perform quick sort
    quick_sort(arr, 0, len(arr) - 1)

    print("Sorted array:", arr)

# Run the main function
if __name__ == "__main__":
    main()

```

16 Implement construction of binary search trees.

```

# Define the structure for a Node in the Binary Search Tree
class Node:
    def __init__(self, value):
        self.value = value # Node value
        self.left = None # Left child
        self.right = None # Right child

# Define the Binary Search Tree
class BST:
    def __init__(self):
        self.root = None # Initialize the root to None

    # Insert a new node into the BST
    def insert(self, value):
        # Call the recursive helper function to insert a value
        if self.root is None:
            self.root = Node(value) # Create the root node if empty
        else:
            self._insert_recursive(self.root, value)

    # Recursive function to insert a node

```

```

def _insert_recursive(self, current_node, value):
    if value < current_node.value:
        # Insert into the left subtree
        if current_node.left is None:
            current_node.left = Node(value)
        else:
            self._insert_recursive(current_node.left, value)
    elif value > current_node.value:
        # Insert into the right subtree
        if current_node.right is None:
            current_node.right = Node(value)
        else:
            self._insert_recursive(current_node.right, value)
    # If value == current_node.value, do nothing (no duplicates in BST)

# In-order traversal (Left, Root, Right)
def in_order_traversal(self):
    result = []
    self._in_order_recursive(self.root, result)
    return result

def _in_order_recursive(self, node, result):
    if node:
        self._in_order_recursive(node.left, result) # Traverse left subtree
        result.append(node.value) # Visit node
        self._in_order_recursive(node.right, result) # Traverse right subtree

# Pre-order traversal (Root, Left, Right)
def pre_order_traversal(self):
    result = []
    self._pre_order_recursive(self.root, result)
    return result

def _pre_order_recursive(self, node, result):
    if node:
        result.append(node.value) # Visit node
        self._pre_order_recursive(node.left, result) # Traverse left subtree
        self._pre_order_recursive(node.right, result) # Traverse right subtree

# Post-order traversal (Left, Right, Root)
def post_order_traversal(self):
    result = []
    self._post_order_recursive(self.root, result)
    return result

def _post_order_recursive(self, node, result):
    if node:
        self._post_order_recursive(node.left, result) # Traverse left subtree
        self._post_order_recursive(node.right, result) # Traverse right subtree
        result.append(node.value) # Visit node

# Main function to test the BST implementation
def main():
    bst = BST()

    # Insert elements into the BST
    elements = [50, 30, 70, 20, 40, 60, 80]
    for elem in elements:
        bst.insert(elem)

```

	<pre> # Perform in-order traversal print("In-order Traversal (Sorted Order):", bst.in_order_traversal()) # Perform pre-order traversal print("Pre-order Traversal:", bst.pre_order_traversal()) # Perform post-order traversal print("Post-order Traversal:", bst.post_order_traversal()) # Run the main function if __name__ == "__main__": main() </pre>
17	Write a menu driven program to perform following operation on Binary Search Tree: a. Create a BST. b. Insert an element in BST. c. Pre-order traversal of BST. d. In-order traversal of BST. e. Post-order traversal of BST. f. Delete an element from BST. <pre> # Define the structure for a Node in the Binary Search Tree class Node: def __init__(self, value): self.value = value # Node value self.left = None # Left child self.right = None # Right child # Define the Binary Search Tree class BST: def __init__(self): self.root = None # Initialize the root to None # Insert a new node into the BST def insert(self, value): if self.root is None: self.root = Node(value) # Create the root node if empty else: self._insert_recursive(self.root, value) def _insert_recursive(self, current_node, value): if value < current_node.value: if current_node.left is None: current_node.left = Node(value) else: self._insert_recursive(current_node.left, value) elif value > current_node.value: if current_node.right is None: current_node.right = Node(value) else: self._insert_recursive(current_node.right, value) # Delete a node from the BST def delete(self, value): self.root = self._delete_recursive(self.root, value) def _delete_recursive(self, node, value): if node is None: return node if value < node.value: node.left = self._delete_recursive(node.left, value) </pre>


```

elif value > node.value:
    node.right = self._delete_recursive(node.right, value)
else:
    # Node with only one child or no child
    if node.left is None:
        return node.right
    elif node.right is None:
        return node.left

    # Node with two children: get the inorder successor (smallest in the right subtree)
    min_val_node = self._min_value_node(node.right)
    node.value = min_val_node.value
    node.right = self._delete_recursive(node.right, min_val_node.value)

return node

```

```

def _min_value_node(self, node):
    current = node
    while current.left is not None:
        current = current.left
    return current

```

```

# In-order traversal (Left, Root, Right)
def in_order_traversal(self):
    result = []
    self._in_order_recursive(self.root, result)
    return result

```

```

def _in_order_recursive(self, node, result):
    if node:
        self._in_order_recursive(node.left, result)
        result.append(node.value)
        self._in_order_recursive(node.right, result)

```

```

# Pre-order traversal (Root, Left, Right)
def pre_order_traversal(self):
    result = []
    self._pre_order_recursive(self.root, result)
    return result

```

```

def _pre_order_recursive(self, node, result):
    if node:
        result.append(node.value)
        self._pre_order_recursive(node.left, result)
        self._pre_order_recursive(node.right, result)

```

```

# Post-order traversal (Left, Right, Root)
def post_order_traversal(self):
    result = []
    self._post_order_recursive(self.root, result)
    return result

```

```

def _post_order_recursive(self, node, result):
    if node:
        self._post_order_recursive(node.left, result)
        self._post_order_recursive(node.right, result)
        result.append(node.value)

```

Menu-driven program to perform BST operations

	<pre> def menu(): bst = BST() # Create an instance of the Binary Search Tree while True: print("\n--- Binary Search Tree Menu ---") print("1. Create a BST") print("2. Insert an element") print("3. Pre-order traversal") print("4. In-order traversal") print("5. Post-order traversal") print("6. Delete an element") print("7. Exit") choice = int(input("Enter your choice: ")) if choice == 1: print("BST Created.") elif choice == 2: value = int(input("Enter the value to insert: ")) bst.insert(value) print(f"Inserted {value} into the BST.") elif choice == 3: print("Pre-order Traversal:", bst.pre_order_traversal()) elif choice == 4: print("In-order Traversal:", bst.in_order_traversal()) elif choice == 5: print("Post-order Traversal:", bst.post_order_traversal()) elif choice == 6: value = int(input("Enter the value to delete: ")) bst.delete(value) print(f"Deleted {value} from the BST (if present).") elif choice == 7: print("Exiting...") break else: print("Invalid choice! Please try again.") # Run the menu function if __name__ == "__main__": menu() </pre>
F,FI, GC D	<pre> import math # Function to calculate the Fibonacci sequence up to the nth number def fibonacci(n): fib_sequence = [] a, b = 0, 1 for _ in range(n): fib_sequence.append(a) a, b = b, a + b return fib_sequence # Function to calculate the GCD of two numbers using Euclidean Algorithm def gcd(a, b): while b: a, b = b, a % b return a # Function to calculate the factorial of a number </pre>

```

def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

# Menu-driven program to select the required operation
def menu():
    while True:
        print("\n--- Menu ---")
        print("1. Fibonacci Sequence")
        print("2. GCD of two numbers")
        print("3. Factorial of a number")
        print("4. Exit")

        choice = int(input("Enter your choice: "))

        if choice == 1:
            n = int(input("Enter the number of Fibonacci numbers to generate: "))
            if n >= 0:
                print(f"Fibonacci sequence up to {n} terms:", fibonacci(n))
            else:
                print("Please enter a non-negative integer.")

        elif choice == 2:
            a = int(input("Enter the first number: "))
            b = int(input("Enter the second number: "))
            print(f"GCD of {a} and {b} is:", gcd(a, b))

        elif choice == 3:
            n = int(input("Enter a number to find its factorial: "))
            if n >= 0:
                print(f"Factorial of {n} is:", factorial(n))
            else:
                print("Please enter a non-negative integer.")

        elif choice == 4:
            print("Exiting...")
            break

        else:
            print("Invalid choice! Please try again.")

# Run the menu function
if __name__ == "__main__":
    menu()

```